



Internship Program Introduction Session

TKM Engineering Students · May 2026

60 minutes · cixio.ai · cixio.com

Today's Session

- Welcome & introduction
- The Car Analogy — scale & complexity
- The Web & App World
- What is CIXIO
- Your role as an intern
- Next steps & study list

Part 1

The Car Analogy

A Small Car Repair Shop

2–3 mechanics — one person does everything

Diagnose, order parts, fix, test, invoice — all one person

Tools: spanners, screwdrivers, OBD scanner

Knowledge lives in their heads — no documentation

If the mechanic is sick → shop closes

Cannot scale beyond 2–3 cars per day

→ **This is a MONOLITH. One system doing everything.**

Fine for small scale. Does not work at large scale.

A Car Manufacturer (Tata Motors, Toyota)

A modern car has ~30,000 individual components

No single person knows all 30,000 parts

Engine team → cylinders, pistons, combustion

Electrical team → wiring harness, sensors, ECU

Software team → infotainment, safety systems

Body team → frame, doors, aerodynamics

QA team → tests every component at every stage

Supply chain → 1,000+ suppliers, globally

Each team builds their component independently.

They hand it off at a defined interface — then it gets assembled.

→ **This is MICROSERVICES.**

A Modern Car is 40% Software

Tesla → more lines of code than a fighter jet

BMW 7-series → 100 million+ lines of code

A modern car has 100+ ECUs (small computers)

Engine management, ABS, airbags, infotainment, autopilot —
all software

Key insight for engineers:

The difference between a small garage and Toyota is not skill.

It is **PROCESS, SPECIALISATION, and TOOLING.**

Right Tool for the Right Job

Engine team needs precision machining tools and heat-resistant alloys.

Body team needs sheet metal presses and welding robots.

You would not use a welding robot to assemble a fuel injector.

The same rule applies in software:

Python → AI / Machine Learning (best ecosystem in the world)

JavaScript / React → Browser UI (browser speaks JavaScript natively)

Flutter / Dart → Mobile apps (one codebase → iOS + Android)

Docker → Packaging & deployment (runs the same everywhere)

Using one language for everything is like building a car from one material.

Part 2

The Web & App World

A Small Website

1 developer, built in a weekend

HTML, CSS, a contact form

Hosted on shared server — ₹100/month

50 visitors/day

If it breaks — fix it manually

Behind the scene: Almost nothing.

The server reads the HTML file and sends it. Done.

What Happens When You Log Into CIXIO

1. Your browser hits our load balancer at app.cixio.ai
2. Load balancer routes to the right container (service)
3. SSO service (sso.cixio.ai) verifies your identity
4. You get a JWT — a signed digital pass for all services
5. AI assistant sends your message to our private LLM gateway
6. Gateway checks: is this safe? Does it contain personal data?
7. AI model runs privately on our own server — not OpenAI
8. Response streams back, token by token, to your browser
9. A compliance receipt is created — cryptographically signed
10. Prometheus logs metrics. Grafana shows dashboards.

All of that — under 2 seconds. Automated. Nobody clicked a button.

Google / Microsoft Scale

1 billion+ users per day

100,000+ engineers — 1,000 separate teams

When you search on Google, in < 200ms:

DNS resolution → Load balancer → Auth check

Search index (petabytes, 1,000s of servers) → ML ranking

Ad auction runs in parallel → Cache check

Response compressed + sent back

Everything logged for analytics and abuse detection

When you use an app, you see 5% of what is actually happening.

The other 95% is infrastructure, security, pipelines, monitoring.

The Same Pattern — Always

Small Scale

One person or one system

One tool for everything

Manual processes

Knowledge in heads

Breaks when one person is absent

Fast to start

Examples:

→ Car repair shop

→ ₹100/month website

→ Single-file Python script

Large Scale

Specialised teams

Right tool for each job

Automated processes

Knowledge in docs and code

Continues when anyone is absent

Fast to scale

Examples:

→ Tata Motors / Toyota

→ Google / Microsoft

→ CIXIO (building toward this)

Part 3

What is CIXIO

The Problem We Are Solving

Three real situations:

Hospital (Germany) → wants AI for doctors' notes

Patient data cannot leave Germany (GDPR). ChatGPT is blocked. Back to manual typing.

Private Bank → wants AI assistant for relationship managers

Client financial data is confidential. Cannot send it to OpenAI.

Defence Contractor → wants AI for code review & documentation

Air-gap requirements. Their code never touches public networks.

All three have the same problem: they cannot use cloud AI.

CIXIO is the answer — AI that runs inside their own building.

What We Have Built (Running in Production)

AI — The brain

cixio-llm · Private LLM gateway — runs AI models on our own server
cixio-ai · Document Q&A, RAG, AI platform
assistant.cixio.ai · Private ChatGPT-like interface

Security & Identity

cixio-sso · Single sign-on — one login for everything
vault, files, scan, token · Document vault, file protection, scanning

Developer & Mobile

cixio-code · VS Code extension + web app (code.cixio.ai)
cixio-mobile-app · Flutter app for iOS + Android

Infrastructure & Monitoring

AWS EC2 (Mumbai) · Docker · GitHub Actions CI/CD · Prometheus + Grafana

Why Do We Use So Many Tech Stacks?

Because each layer has different requirements — just like the car.

Python + FastAPI	→ AI services	(Python dominates AI/ML)
Next.js + React	→ Web frontend	(browser-native, fast SSR)
Flutter + Dart	→ Mobile app	(one code → iOS + Android)
Node.js + TypeScript	→ Background APIs	(event-driven, fast JSON)
MongoDB	→ Flexible data	(JSON documents, no rigid schema)
PostgreSQL	→ Analytics & SQL	(structured queries, reporting)
Redis	→ Speed layer	(in-memory, microsecond reads)
Docker	→ Packaging	(runs the same on every machine)
GitHub Actions	→ CI/CD	(push code → auto-deploy)
AWS Mumbai	→ Cloud infra	(EC2, ALB, Route53, SSM)

You don't use a welding robot to assemble a fuel injector.

Part 4

Your Role as an Intern

5 Internship Tracks — Pick Based on Your Interest

Track A · Frontend / Web

Next.js, React, TypeScript, Tailwind CSS

Track B · Mobile

Flutter, Dart — iOS + Android from one codebase

Track C · Backend / API

Python + FastAPI or Node.js + TypeScript

Track D · AI / LLM

Python, Ollama, LanceDB, RAG pipelines, prompt engineering

Track E · Infrastructure / DevOps

Docker, GitHub Actions, AWS, Terraform

What You Will Have Learned After 3 Months

Git professionally — branches, PRs, code review, commit conventions

Docker — build, run, and understand containerised applications

REST APIs — build and consume an API with auth and error handling

CI/CD — push code and watch it automatically deploy to production

Cloud basics — understand AWS EC2, load balancers, DNS, secrets

Real codebase navigation — read, understand, extend production code

Most 4th-year students and fresh graduates do not have these skills.

You will leave this internship ahead of your peers.

What we look for above skill: Attitude. Can you figure things out?

Career Path

Intern (3 months)

Real features, real repositories, real production deployments



Trainee Engineer (6–12 months)

Assigned product area, own features end-to-end



Junior Engineer → Engineer (full-time)

Design, architect, mentor new interns

We prefer to grow people internally. If you put in the work — you will have a real shot.

Next Steps — If You Are Interested

1. Go through TRAINEE-STUDY-LIST

Every concept you will see in the first few weeks — explained in plain English.

Suggested order: $A \rightarrow B \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

2. Pick 3 terms from the list

1 term you found interesting. 1 term you found confusing. 1 you already knew.

3. Send us an email

Reach out **Manu Sir**